Министерство цифрового развития, связи и массовых коммуникаций Российской Федерации федеральное государственное бюджетное образовательное учреждение высшего образования
«Сибирский государственный университет телекоммуникаций и информатики»

Кафедра вычислительных систем

ОТЧЕТ
по практической работе 7

по дисциплине **«Основы разработки на Java»**

Выполнил:
студент гр. ИС-542
«25» октября 2026 г. _____ /Горбачев Д.В./

Проверил:
Ассистент кафедры ВС
«25» октября 2026 г. _____ /Гузенков С.К./

Оценка «_____»

Новосибирск 2026

Ссылка на репозиторий: _____

В работе реализованы чтение и запись каталога курсов, безопасное управление файловыми ресурсами, текстовый и бинарный импорт, потоковая обработка и проверка версий данных.

Задание 1 Сохранение каталога в UTF 8

Сохранить каталог курсов в UTF-8 через Files.writeString, прочитать его через Files.readString и обработать отсутствующий файл и каталог вместо файла.

В Task01FilesReadWrite текст каталога записывается и читается с явным указанием StandardCharsets.UTF_8. Для отсутствующего файла ожидается NoSuchFileException. Чтение и запись по пути существующего каталога проверяются отдельно и должны завершиться IOException. Другие ошибки ввода-вывода не скрываются.

Пробный каталог успешно сохранён и прочитан. Отсутствующий файл и путь-каталог отклонены ожидаемыми ошибками; код не сообщает об успешной операции после неудачной записи.

Задание 2 Буферное чтение и запись

Переписать текстовую обработку на BufferedReader и BufferedWriter, подсчитать строки и символы и объяснить переход от байтов к символам.

Task02BufferedReaderWriter использует буферные потоки и UTF-8. BufferedReader получает символы из декодера, который преобразует байты файла в символы Java; BufferedWriter выполняет обратное преобразование при записи. Код подсчитывает строки отдельно, а codePointCount считает символы Unicode, в том числе корректно для символов вне базовой многоязычной плоскости.

В демонстрационном каталоге прочитаны две строки и подсчитаны 38 символов Unicode. Весь текст записывается буферным writer и читается построчно.

Задание 3 Закрытие ресурсов при ошибке

Создать конвейер чтения и записи, вызвать исключение в середине обработки и проверить закрытие обоих ресурсов и порядок вызовов close.

В Task03TryWithResources ресурсы объявлены в try-with-resources: сначала Reader, затем Writer. После чтения и записи намеренно возникает IOException. Обёртки ресурсов записывают факт закрытия в список, поэтому

проверяется не только обработка исключения, но и реальное закрытие каждого объекта.

Java закрыла ресурсы в обратном порядке объявления: сначала writer, затем reader. Получен ожидаемый порядок [writer, reader], оба ресурса закрыты даже после исключения.

Задание 4 Ленивые строки Files.lines

Использовать Files.lines внутри try-with-resources, показать ошибку при возврате закрытого потока и объяснить, кто владеет ресурсом.

Безопасный метод countLines обрабатывает stream и закрывает его внутри try-with-resources. В returnClosedStream тот же stream возвращается из блока, но ресурс закрывается до того, как вызывающий код начнёт чтение. Попытка вызвать count для возвращённого потока приводит к IllegalStateException.

Файловый stream должен использоваться внутри области владения открывшим его методом. Если метод передаёт открытый stream вызывающему коду, обязанность закрыть его должна быть явно передана вызывающей стороне; здесь показан небезопасный вариант с уже закрытым stream.

Задание 5 Бинарный формат каталога

Записать и прочитать записи через DataOutputStream и DataInputStream, включив magic header, версию и число записей; отклонять неизвестную версию и обрезанный файл.

CourseBinaryStore записывает сигнатуру, номер версии, количество курсов и поля каждой записи: ID, название, длительность и пройденные часы. При чтении проверяются сигнатура, поддерживаемая версия, допустимое количество записей, полнота каждой записи и отсутствие лишних байтов. EOF при чтении записи переводится в понятную ошибку обрезанного файла.

Проверен круговой проход одного курса. Файл с неизвестной версией и файл, обрезанный в конце записи, отклоняются; неполный каталог не возвращается вызывающему коду.

Задание 6 Безопасная замена файла

Сначала записать новый каталог во временный файл того же каталога, закрыть его и заменить целевой файл move-операцией с обработкой отсутствия atomic move.

AtomicCourseTextStore создаёт временный файл рядом с целевым, полностью записывает данные, вызывает flush и закрывает writer до перемещения. Затем выполняется замена с ATOMIC_MOVE и REPLACE_EXISTING. Если файловая система не поддерживает атомарное перемещение, перехватывается AtomicMoveNotSupportedException и используется обычная замена.

В текущем запуске файловая система поддержала atomic move. В коде предусмотрен fallback, но в этой среде он не срабатывал. Целевой файл был заменён только после завершения записи временного файла.

Задание 7 Импорт без частичной публикации

Собрать ошибки повреждённого импорта с номерами строк и выбрать контракт, который не публикует половину каталога.

CatalogImporter читает файл построчно, временно собирает корректные курсы и накапливает ImportError с номером каждой ошибочной строки. Контракт задан типом CatalogImportResult: Success содержит готовый неизменяемый список, а Failure содержит только ошибки. Если найдена хотя бы одна ошибка, список курсов наружу не публикуется.

В тестовом файле ошибочна строка 2. Импорт вернул Failure с номером строки и сообщением, а не каталог из первой и третьей записей. Выбрана политика all-or-nothing.

Задание 8 Потоковый импорт большого файла

Обрабатывать файл без readAllLines, вести счётчики, ограничить сохранённый список ошибок и отменять импорт после заданного порога.

StreamingCatalogImporter читает по одной строке и не хранит весь список Course в памяти. Корректные строки сразу записываются во временный файл, ошибки учитываются счётчиком, а в отчёте сохраняется только заданное число сообщений. При достижении порога чтение прекращается, временный файл удаляется и прежний целевой каталог остаётся на месте.

Контракт потокового импорта допускает публикацию корректных строк при ошибках, если отмена не наступила; это частичный импорт, в отличие от задания 7. В демонстрации из пяти строк три корректны, две ошибочны, но хранится только одно сообщение. Отдельный запуск достигает порога отмены и подтверждает, что прежний каталог не заменён.

Задание 9 Версии схемы и контрольная сумма

Добавить версию схемы и checksum, выполнить миграцию версии 1 в версию 2, отклонять будущие версии и обнаруживать повреждение до публикации объектов.

VersionedCourseStore использует SHA-256 для проверки содержимого до его разбора. Версия 1 не содержит поля completedHours; при миграции оно получает значение 0. Версия 2 хранит это поле. Неподдерживаемая будущая версия отвергается сразу после чтения заголовка, а несовпадение checksum приводит к ошибке до создания возвращаемого списка курсов.

Проверены сохранение прогресса в версии 2, чтение версии 1 с миграцией, отказ от будущей версии и обнаружение изменённого байта. Повреждённые данные не возвращаются как каталог.

Запуск gradle run собрал проект и выполнил демонстрации всех девяти заданий без ошибок.